

# Kronos: The Architecture of a Multi-Tenant WebAssembly Runtime

John Abrahams  
University of Pennsylvania  
September 2026

**Abstract**—Kronos is a multi-tenant WebAssembly runtime built on Wasmtime. It runs untrusted programs from many tenants in one host process. Before a module is instantiated, Kronos compares its imports with the tenant’s capabilities and rejects the module if it imports a file or network function the tenant was not granted. No code from a rejected module runs. Each invocation runs in a new store limited by fuel and an epoch-based deadline. Around the runtime, Kronos provides a control plane with tenants and shareable applications, a launcher that runs Python, JavaScript, Ruby, and PHP as unmodified WASI modules, host intrinsics and a thread-spawn handler for data-parallel guests, and an interface through which AI agents deploy and run code. This paper describes the architecture of the final research version (July 2026), reports measurements of memory density, throughput, and threaded execution, and analyzes which isolation and resource properties the code enforced.

**Index Terms**—WebAssembly, multi-tenancy, sandboxing, capabilities, Wasmtime, AI agents

## I. INTRODUCTION

Services that run untrusted code, including programs written by language models, usually place that code in a container or microVM and restrict it with separate policy: seccomp filters, network namespaces, and egress rules [1]–[5]. The guest starts with the full Linux system-call interface, and the operator removes access from it. A missing rule leaves access in place, and language models can find and exploit such gaps [6].

A WebAssembly module can reach the host only through the functions it imports, and it must declare each import before it is instantiated [7], [8]. WASI defines a system interface on the same principle, without ambient authority [9]. A host can therefore list every host function a module may call before the module runs.

Kronos is a multi-tenant WebAssembly runtime written in Rust on Wasmtime [11]. It uses the import list for access control. Each tenant holds a set of capabilities. Before instantiating a module, Kronos compares the module’s imports with those capabilities and rejects the module if any import requires a capability the tenant lacks. Authorization is decided once, from the declared interface, before any guest code runs. This paper calls the policy *deny by construction*. Each invocation then runs in a new Wasmtime store with a fuel budget and a wall-clock deadline, and the store is discarded when the call returns.

Kronos is described as a *Type II exovisor*. It runs as an ordinary process on a host operating system, as a hosted (Type II) hypervisor does. Like an exokernel [10], it exposes a small, fixed set of primitives instead of a full operating-system

interface. Unlike an exokernel, which exposes hardware, it exposes file descriptors and C-library functions.

This paper describes Kronos at commit 2b73f3cb8 (July 27, 2026), the last version before the research runtime was removed from the code base. Its contributions are:

- the runtime design: engine configuration, linkers, the per-invocation store lifecycle, and the import surface (Section IV);
- the capability model and its check at instantiation (Section V);
- resource control with fuel and epoch deadlines, and its limits for host calls (Section VI);
- host intrinsics for data-parallel C++ and a thread-spawn handler that confines thread faults to one isolate (Section VII);
- the control plane: tenants, module and application admission, a launcher for interpreted languages, and interfaces for AI agents (Section VIII);
- measurements of memory density, throughput, and threaded execution (Section IX), and an analysis of which properties the code enforced (Section X).

Where source comments disagree with the code, this paper follows the code.

## II. BACKGROUND AND THREAT MODEL

### A. WebAssembly and Wasmtime

A WebAssembly module declares every function, memory, table, and global it needs from its environment. Instantiation binds each declaration to a definition from the embedder. Guest code cannot address memory outside its linear memory, and its control flow is structured, so the import list bounds what a module can ask the host to do [7].

Wasmtime [11] compiles modules ahead of time with Cranelift. Its API has four central objects. An Engine holds compilation settings, compiled code, and a global *epoch* counter. A Store holds instance state and cannot be used by two threads at once. A Linker maps import names to host definitions. An InstancePre is a module whose imports are already resolved.

Wasmtime provides two execution budgets. *Fuel* decreases as guest code runs, and the guest traps when it reaches zero. Under *epoch interruption*, compiled code compares the engine epoch with its store’s deadline at function entries and loop back-edges, and traps once the deadline has passed.

WASI preview 1 defines 46 functions for files, clocks, randomness, sockets, and process control. A guest can open files only under directories the embedder preopens [9].

### B. Threat model

The adversary is the guest module. Its code is arbitrary, may be written by a language-model agent or designed to attack the host, and runs without human review. The host process holds state the guest must not reach: credentials for external services, other tenants’ stores, and any file or network destination the tenant was not granted. The trusted computing base is the Kronos process, Wasmtime and its generated code, and the host operating system. The design targets three properties.

**P1 No unauthorized effects.** A module that imports a file or network function its tenant was not granted does not run.

**P2 Tenant independence.** A trap, panic, or timeout in one tenant’s invocation does not change the outcome of another tenant’s invocation.

**P3 Bounded work.** Every invocation ends within its fuel and wall-clock budgets.

Microarchitectural side channels [22] and deliberate exhaustion of shared host resources [21] are out of scope. Section X notes resource paths that the budgets do not cover.

## III. SYSTEM OVERVIEW

Kronos is a single Rust process with a control plane and an execution plane (Fig. 1). All clients use one HTTP API. The web interface and the `khronos` command-line tool call it directly. AI agents call it through `kronos-mcp`, a Model Context Protocol server that turns tool calls into HTTP requests.

The control plane authenticates each request and maps it to a tenant. It manages tenants, organizations, uploaded modules, applications, schedules, connectors, and a pool of language-model tokens. State is stored in SQLite, and application bundles are stored in a host directory.

The execution plane holds two instances of the runtime library. The *module runtime* runs raw WebAssembly modules through `/invoke`, with a two-second deadline and  $10^{10}$  units of fuel per call. The *launcher runtime* runs programs in Python, JavaScript, Ruby, PHP, and compiled languages as WASI command modules, with a 120-second deadline and no fuel limit. Each runtime owns one Wasmtime engine and one epoch-ticker thread.

Figure 2 shows the code structure. `khronos-runtime` contains all Wasmtime code, about 3,700 lines of Rust. `khronos-launcher` is a language registry built on the runtime. `khronos-server` implements the HTTP API with `axum` 0.7 and `Tokio` and stores state with `rusqlite`. The command-line tool links the runtime and launcher for local runs. `kronos-mcp` depends on no other crate. The workspace uses Wasmtime 29.0.1.

Every execution follows four steps. The control plane takes the request’s authority from the tenant record or the application’s capability profile. The runtime checks the module’s imports against that authority. A store is created with fuel

TABLE I  
WASMTIME CONFIGURATION OF EACH RUNTIME.

| Setting              | Value                            |
|----------------------|----------------------------------|
| Compiler             | Craneflight, optimized for speed |
| Fuel consumption     | enabled                          |
| Epoch interruption   | enabled                          |
| Parallel compilation | enabled                          |
| Reference types      | enabled                          |
| Multiple memories    | disabled                         |
| Linear memory        | guard pages (default)            |
| Pooling allocator    | optional, disabled by default    |

and a deadline, and the guest runs. The store is then dropped. Sections IV–VII cover the runtime, and Section VIII covers the control plane.

## IV. RUNTIME

Kronos relies on Wasmtime for compilation, memory protection, fuel accounting, and traps. The runtime adds four components: a tiered import surface behind a capability check, a store lifecycle with per-call budgets, host intrinsics for data-parallel code, and a thread-spawn handler for a process shared by tenants.

### A. Engine configuration

Each Runtime creates one engine with the settings in Table I. Parallel, speed-optimized compilation shortens the compilation of large modules such as the 21 MB WASI build of CPython. Guard-page linear memory, the Wasmtime default on 64-bit hosts, removes software bounds checks from compiled code. The optional pooling allocator reserves one instance, memory, table, and stack per engine. It lowers instantiation time at the cost of memory reserved in advance.

Each runtime also starts a thread that advances the engine epoch every millisecond. A deadline  $d$  becomes  $\lceil d/1 \text{ ms} \rceil$  ticks, so the default two-second deadline is 2,000 ticks. Each check costs the guest one load, one comparison, and one branch.

The engine shares compiled code and the epoch counter among all its stores. Source comments give two conflicting models, one runtime per process and one per tenant, and the crate enforces neither. The server uses one runtime per execution path for all tenants. The benchmark harnesses use one runtime per tenant. Section X-D discusses the effect of this choice.

### B. Linkers and compilation

Each runtime builds two linkers at start-up. Both register all Kronos host functions, and the second also registers the WASI preview 1 functions. Registration therefore happens once per runtime. Compiling a module takes four steps.

- 1) Wasmtime compiles the module from text or binary form.
- 2) Both linkers are cloned, and every import without a definition becomes a *trap stub*: a function of the import’s type that traps when called. C++ built with clang imports exception-handling symbols that exception-free programs

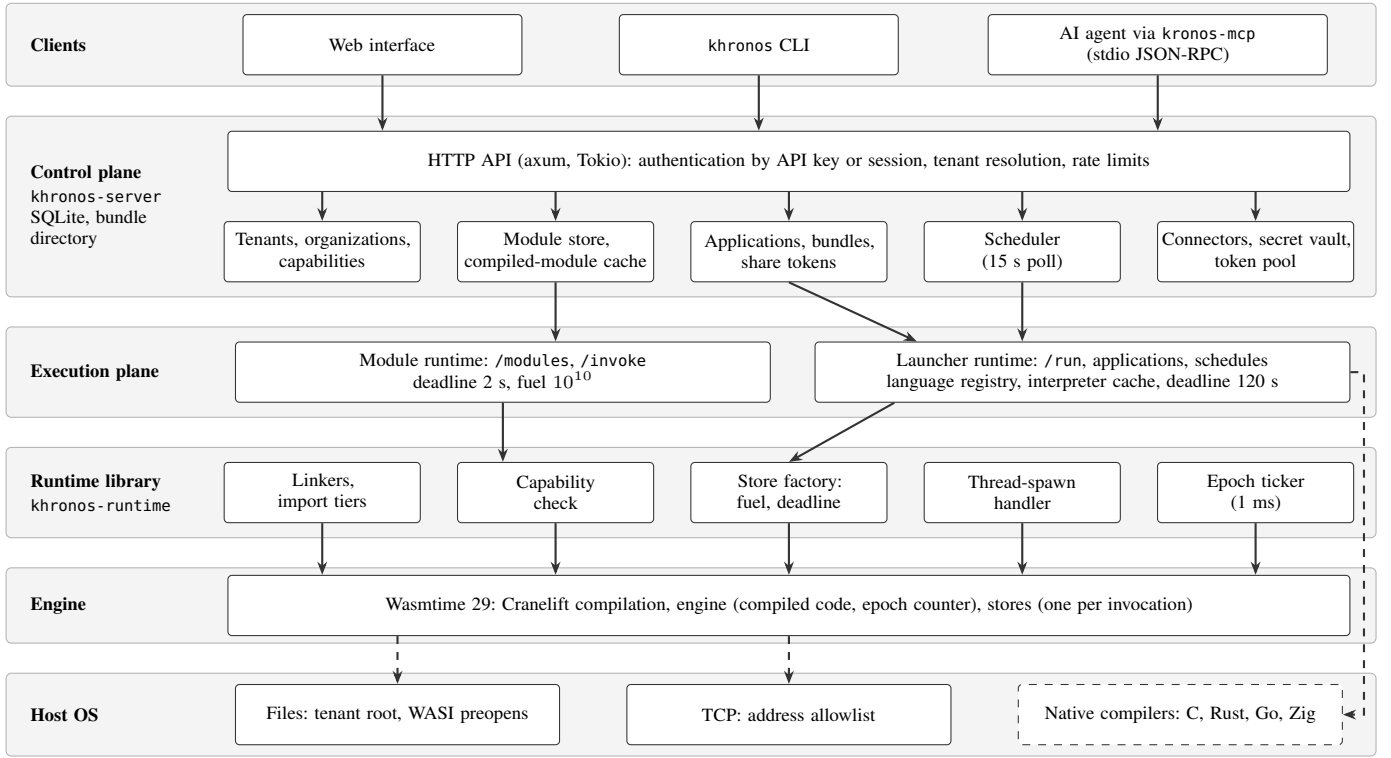


Fig. 1. Layered architecture of Kronos. Solid arrows are requests and calls; dashed arrows are access to host resources. The native compilers run outside the sandbox.

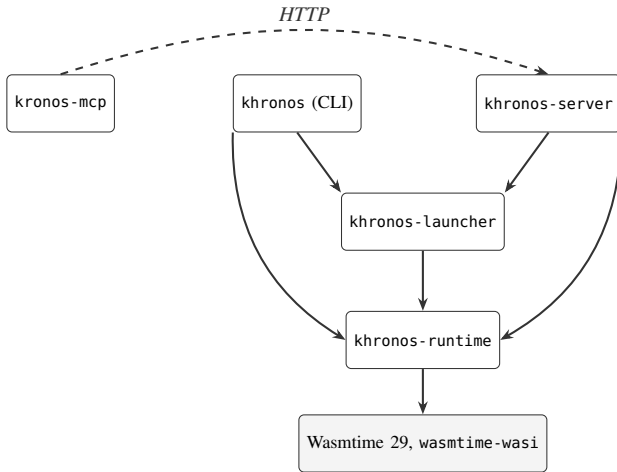


Fig. 2. Crate dependencies. kronos-mcp reaches the server only over HTTP.

never call. The stubs let such modules instantiate and fail only if a missing function is called.

- 3) If all imports are functions, an InstancePre is built for each linker, so later instantiations skip import resolution.
- 4) The runtime records whether the module uses WASI and whether it imports its memory, stack pointer, or function table. Object files built with `clang -c` import these three and expect the host to supply them.

TABLE II  
PER-INVOCATION STORE DATA.

| Field                   | Contents                                            |
|-------------------------|-----------------------------------------------------|
| limits                  | Capabilities, budgets, file root, network allowlist |
| wasi                    | WASI context, if granted                            |
| host_memory             | Guest memory used by file and network handlers      |
| host_indirect_table     | Function table used by intrinsics                   |
| host_stack_pointer      | Host-supplied stack pointer                         |
| heap_base               | Bump pointer for operator new                       |
| aborted, trap_message   | Abort flag and first error, shared with threads     |
| fd_table                | Descriptors opened by host handlers                 |
| tenant_id, token_budget | Tenant identity and token pool                      |

### C. Store state

Each invocation uses a new store holding the data in Table II. Dropping the store closes its files and sockets and releases its WASI context. No runtime state carries over between invocations.

### D. Instantiation

Figure 3 shows the checked instantiation path.

- 1) **WASI.** A module that imports WASI is rejected if no WASI context was supplied. Otherwise its trap stubs would let it instantiate.

- 2) *Capabilities*. The runtime computes the effective capability set and runs the check in Section V.
- 3) *Limits*. The limits are copied into the store. An explicit network allowlist replaces the configured one. The allowlist is cleared when networking is not granted.
- 4) *Budgets*. The store is created with its fuel and epoch deadline. The deadline must be set first, because a store without a deadline traps at once under epoch interruption.
- 5) *ABI objects*. If the module imports them, the host defines a linear memory, a stack pointer starting at 1 MiB, and a function table of at least 128 entries.
- 6) *Instantiation*. Wasmtime links the module and runs its start function.
- 7) *Handles*. The runtime records which memory the host handlers use and resets an exported stack pointer to 1 MiB.
- 8) *Call*. The export is looked up, the argument count is checked, and the function is called. Any failure, including fuel exhaustion and deadline expiry, returns as a trap.

#### E. Import surface

A tenant module can import 56 host functions outside WASI. A module granted a WASI context can also import the 46 WASI preview 1 functions (Fig. 4, Table III). The tier names come from the Go runtime that preceded the Rust version.

The clock always returns zero, so guests cannot use time as a timing channel or a source of nondeterminism. The WASI-NN functions let modules written for that interface link, but no inference backend was built. Tier 3 holds the only non-WASI functions that touch host resources and is the only group the capability check covers.

### V. CAPABILITY MODEL

#### A. Capabilities and gated names

A capability set has two flags, `AllowFilesystem` and `AllowNetworking`, both false by default. Twenty-two Tier 3 import names are mapped to these flags (Table IV). Nine names have handlers. The other thirteen are listed so that a module importing them is rejected when the capability is missing. When the capability is present, they resolve to trap stubs.

#### B. The check

Figure 5 shows the check in `check_gated_imports`. It visits the module’s imports in order. It skips imports from `wasi_snapshot_preview1` and from any module other than `env`. It classifies each remaining name as open, requiring one capability, or accepting either. It returns `CapabilityDenied` at the first name whose requirement is not met. The check reads declarations only, so a module that imports `connect` is rejected whether or not it calls it. Figure 6 shows a rejection on the server’s module path.

#### C. Placement at instantiation

Because the check runs at instantiation, host handlers never consult capabilities, and calls carry no policy check. The linker defines every handler for every tenant and stubs unknown

names, so the check is the only point of rejection. A rejected module allocates no memory, uses no fuel, and runs no instructions.

#### D. WASI

The check does not classify WASI imports. WASI limits file access to preopened directories, so file authority is set when the WASI context is built [9]. Kronos adds one rule: a module that imports any WASI function needs a WASI context, and the server builds one only for tenants with `AllowFilesystem`. On the module path, this context has standard input and output and no preopened directories.

#### E. Instantiation entry points

The runtime has five instantiation entry points (Table V). The server’s module path uses `instantiate_with_wasi` and `instantiate_without_wasi`, which check the tenant’s capabilities. Section X-B analyzes the other three.

#### F. File and network handlers

1) *Files*: A tenant with `AllowFilesystem` has a root directory, and guest paths resolve beneath it. The resolver strips leading slashes, rejects paths that contain a parent, root, or prefix component with `EACCES`, and joins the rest to the root. If no root is set, every file operation returns `ENOENT`. Resolution is lexical. Paths are not canonicalized, because a new file does not yet exist, and symbolic links are followed. `openat` supports the read, write, create, truncate, and append flags and ignores its directory argument. `stat64` reports only the file size. Descriptors come from a per-store table that starts at 3 and never reuses numbers.

2) *Network*: Only IPv4 TCP is supported; there is no DNS, UDP, or listening socket. `socket` allocates a placeholder descriptor. `connect` reads a `sockaddr_in` from guest memory and checks the address and port against the store’s allowlist before it inspects the descriptor. Any address not on the list returns `ECONNREFUSED`, so an empty list blocks all connections. Allowed connections have a two-second connect timeout and five-second read and write timeouts. `sendto` and `recvfrom` work only on connected sockets, so the allowlist check in `connect` covers all traffic.

### VI. RESOURCE CONTROL

#### A. Fuel

On the module path, each store starts with  $10^{10}$  units of fuel. The server reads the remaining fuel before and after each call and records the difference as `fuel_consumed`. Fuel is thus both a limit and a unit of accounting. The launcher sets fuel to the maximum value, since starting an interpreter uses more instructions than most compute kernels, and relies on its deadline instead.

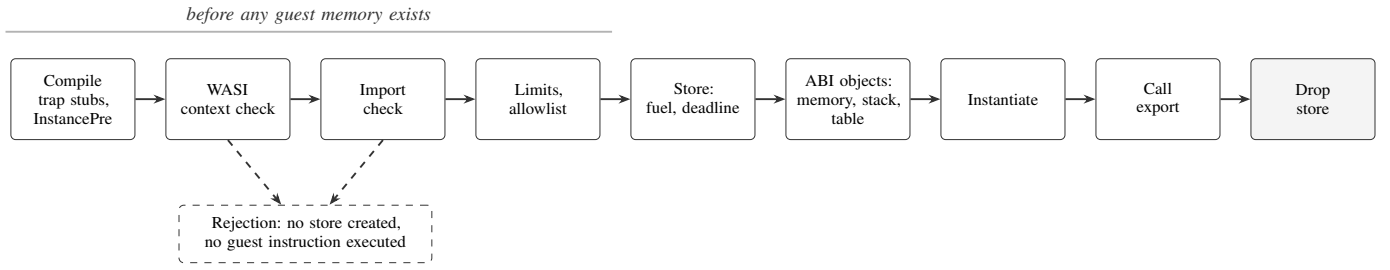


Fig. 3. Checked instantiation path. Compilation happens once per module; the other stages happen once per invocation.

TABLE III  
HOST FUNCTIONS AVAILABLE TO TENANT MODULES.

| Group                  | Count | Functions and behavior                                                                                                                                                                                                                                                   | Gate         |
|------------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| Tier 1: C++ runtime    | 11    | <code>operator new</code> as a bump allocator from address 4096, capped below 1 MiB; <code>operator delete</code> does nothing; exceptions and <code>std::terminate</code> trap                                                                                          | none         |
| Tier 2: builtins       | 26    | Bounds-checked <code>memcpy</code> , <code>memmove</code> , <code>memset</code> ; heap size and growth; <code>abort</code> and failed assertions trap; clock returns zero; double-precision math                                                                         | none         |
| Token budget           | 1     | <code>debit_tokens</code> debits the tenant's share of the token pool                                                                                                                                                                                                    | none         |
| Intrinsics             | 4     | <code>MAP</code> , <code>REDUCTION</code> , and two reducers (Section VII)                                                                                                                                                                                               | none         |
| WASI-NN                | 5     | Inference interface; every call returns "unsupported"                                                                                                                                                                                                                    | none         |
| Tier 3: files, network | 9     | <code>openat</code> , <code>read</code> , <code>write</code> , <code>close</code> , <code>stat64</code> , <code>socket</code> , <code>connect</code> , <code>sendto</code> , <code>recvfrom</code> ; <code>musl</code> convention, negated <code>errno</code> on failure | per name     |
| WASI preview 1         | 46    | <code>fd_write</code> , <code>path_open</code> , <code>clock_time_get</code> , and others                                                                                                                                                                                | WASI context |
| Thread spawn           | 1     | <code>wasi:thread-spawn</code> , threaded modules only                                                                                                                                                                                                                   | none         |

TABLE IV  
GATED IMPORT NAMES AND REQUIRED CAPABILITIES.

| Required capability | Names with handlers                                                                      | Names rejected or stubbed                                                                                                                                                                                  |
|---------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AllowFilesystem     | <code>openat</code> , <code>stat64</code>                                                | <code>unlinkat</code> , <code>lstat64</code> , <code>fstat64</code> , <code>getcwd</code> , <code>ioctl</code> , <code>fcntl64</code> , <code>mkdirat</code> , <code>fd_read</code> , <code>fd_seek</code> |
| AllowNetworking     | <code>socket</code> , <code>connect</code> , <code>sendto</code> , <code>recvfrom</code> | <code>bind</code> , <code>listen</code> , <code>accept4</code> , <code>emscripten_fetch</code>                                                                                                             |
| Either              | <code>close</code> , <code>read</code> , <code>write</code>                              |                                                                                                                                                                                                            |

TABLE V  
INSTANTIATION ENTRY POINTS AND THE CAPABILITIES EACH CHECKS.

| Entry point                                                                | Check | Capabilities applied                                              |
|----------------------------------------------------------------------------|-------|-------------------------------------------------------------------|
| <code>instantiate_with_wasi</code> , <code>instantiate_without_wasi</code> | yes   | Caller's capabilities; used by the server's module path           |
| <code>instantiate</code>                                                   | yes   | Filesystem treated as granted if the module imports WASI          |
| <code>run_command</code>                                                   | yes   | Filesystem treated as granted; networking from the caller         |
| <code>instantiate_threaded</code>                                          | yes   | Filesystem treated as granted; networking from the runtime limits |
| <code>invoke</code> (fast path)                                            | no    | Modules that import only functions skip the check                 |

### B. Deadlines

Deadlines use epoch interruption. A store's deadline is set relative to the engine epoch at instantiation, and the epoch advances every millisecond. No deadline callback is configured, so an expired deadline always causes a trap. The epoch belongs to the engine, so all stores from one runtime share the same counter (Fig. 7).

### C. Memory

The limits set a maximum memory size of 64 MiB on the module path and 1 GiB in the launcher. The runtime applies this

value only through the pooling allocator. The allocator is off by default, and stores have no resource limiter. A guest memory is therefore bounded only by its own declared maximum and by the 4 GiB address space of 32-bit WebAssembly. Memories that the host creates for a module have a minimum of 16 pages and no maximum.

### D. Concurrency

The server limits concurrent module calls per tenant with a semaphore sized by the tenant's `max_concurrency`. A request

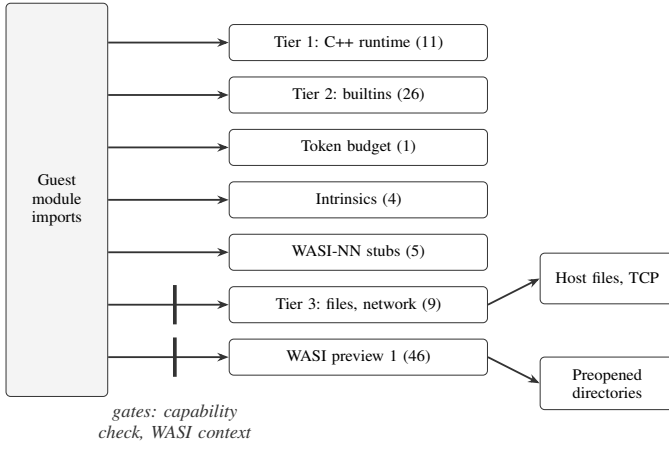


Fig. 4. Import tiers. Only Tier 3 and WASI reach host resources, and only they are gated.

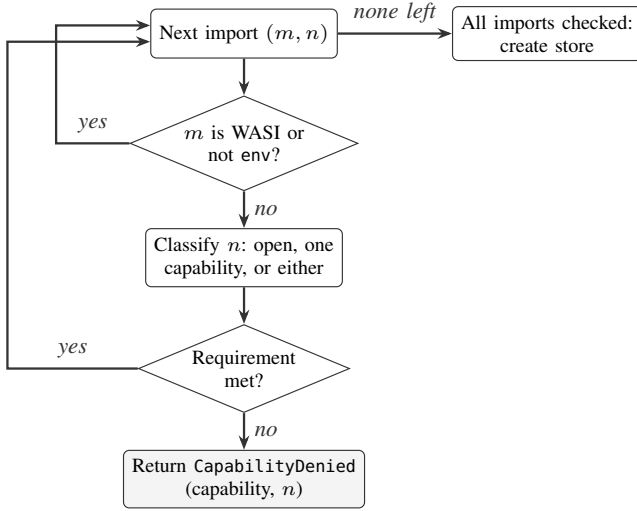


Fig. 5. Import check, performed before the store is created.

```
(module
  (import "env" "__syscall_socket" (func ...))
  (import "env" "__syscall_connect" (func ...))
  (memory (export "memory") 1)
  (func (export "main") ...))

Tenant: AllowFilesystem and AllowNetworking not granted.
POST /invoke → HTTP 403, CapabilityDenied for
AllowNetworking at env::__syscall_socket. No store is created.
```

Fig. 6. Rejection of a module that imports network functions.

that finds no free permit receives HTTP 429 immediately. Launcher runs have no per-tenant limit.

### E. Failure reporting

The runtime reports every error during a call as a trap with Wasmtime’s message. The server classifies the message by substring. “All fuel consumed” becomes `fuel_exhausted`, “epoch deadline” or “interrupt” becomes `deadline`, and any

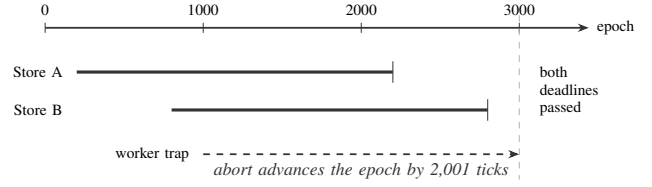


Fig. 7. Engine epoch and store deadlines. Each bar runs from a store’s creation to its deadline, 2,000 ticks later. The abort path of the thread-spawn handler (Section VII) moves the shared counter past both deadlines.

other trap becomes trapped. Once a call starts, the HTTP status is 200, and the body carries the outcome.

### F. Host calls

Fuel and epoch checks apply only to guest code. A host function uses no fuel, and no epoch check runs while the guest waits for a host call to return. Each host function must bound its own work. The network handlers use timeouts; the file handlers have none.

## VII. PARALLEL EXECUTION

### A. Guest interface

Data-parallel guests use `walk`, a C++ library. The programmer supplies identity values and a combining operation and calls a wrapper:

```
reductionWrapper<int,int>(data, n, identity_T,
  identity_R, cilk_reduce, reduction, op)
```

`mapWrapper` is the per-index equivalent. Built without `-pthread`, the wrappers call host intrinsics. Built with `-pthread`, they create threads with `pthread_create`. The guest source is the same in both builds, in the style of Cilk [13].

### B. Host intrinsics

There are four intrinsics: `MAP` and `REDUCTION`, which take  $(start, end, step, p)$ , and two reducers that fold an element or a partial result into an accumulator. Clang erases C++ templates at the WebAssembly boundary, so every intrinsic receives only 32-bit integers. The argument  $p$  points to a `libc++ std::function`. On `wasm32` this object stores its invoker as a function-table index at offset 8 (Fig. 8). The host reads the index, looks it up in the module’s function table, checks its signature, and calls the invoker once for each index in the range, on the caller’s store.

An earlier version ran the invoker calls in parallel on Rayon [14] threads. It copied guest memory for each worker and merged the changes afterward. It was slower than sequential execution, because copying and merging cost more than the work per element, and merging struct accumulators such as  $k$ -means centroids produced wrong results. It was removed on April 28, 2026. Since then the intrinsics run sequentially, and all parallelism comes from guest threads.

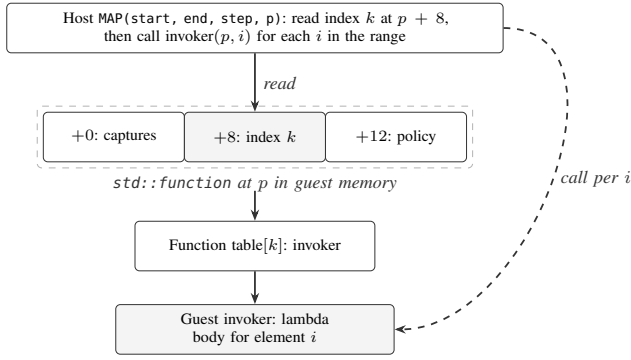


Fig. 8. Dispatch of a host intrinsic to a guest lambda.

### C. Guest threads

Threaded guests target `wasm32-wasi-threads`. They import a shared memory and `wasi:thread-spawn`. `wasi-libc` implements `pthread_create` by allocating a thread stack in shared memory and calling `thread-spawn` with a start argument [15]. A Wasmtime store cannot be shared between threads, so each guest thread needs its own store bound to the shared memory. The `wasmtime-wasi-threads` handler available at the time exited the host process when a spawned thread trapped. Kronos therefore uses its own handler (Fig. 9).

`instantiate_threaded` requires a shared-memory import. It creates the shared memory; defines the memory, a stack pointer, a table, and the spawn handler; and instantiates the main store. It then calls the guest's `_initialize`. Without this call, `wasi-libc`'s thread runtime is not set up and `pthread_join` never returns.

For each spawn, the handler assigns a thread ID, returns it to the guest, and submits a task to Rayon's global pool. The pool caps total parallelism across all tenants. The task runs inside `catch_unwind`. It takes a worker slot from the isolate's pool or builds a new one, which consists of a store, a linker, the shared memory, a stack pointer, a table, and a nested spawn handler. It resets the stack pointer, refills fuel, sets a new deadline, and calls `wasi_thread_start`. On success, the slot returns to the pool.

On a trap or panic, the slot is discarded, since the guest's C library state in it may be inconsistent. The handler records the first error, sets the isolate's abort flag, and advances the engine epoch by one deadline plus one tick. The isolate's other threads then trap at their next epoch check, and the process keeps running. A unit test forces a worker to execute an unreachable instruction and checks that the host survives, the abort flag is set, and the error names the thread.

The walk wrappers use four threads and fall back to sequential execution below 4,096 elements for reductions and 256 for maps. Per-element state must be captured by value, since shared mutable state is a data race. Only fork-join parallelism is supported: threads that wait on each other can deadlock when the Rayon pool is full.

## VIII. CONTROL PLANE

### A. Process model

The server is a Tokio process with one shared state object. It holds the SQLite connection behind a mutex, the two runtimes, an in-memory cache of compiled modules, per-tenant semaphores, a broadcast channel for invocation events, and the token pool. The module cache has no size limit and is not saved to disk, so modules are recompiled from their stored bytes after a restart. Launcher runs `execute` on Tokio's blocking thread pool. Module calls `execute` on the async worker thread that received the request, and only their deadline bounds them.

### B. Principals and tenants

Figure 10 shows the principal model. The *tenant* is the unit of isolation and policy. It owns modules, applications, schedules, secrets, and connector registrations. A tenant record holds the two capability flags, limits on fuel, time, and memory, a concurrency limit, and an administrator flag.

A request maps to a tenant in one of two ways. An API key, 32 random bytes, acts as the tenant's owner. A session identifies a user and the organization the user acts for. An organization's slug is also its tenant ID, and each user gets a personal organization at first sign-in. API keys and session, share, and invitation tokens are stored as SHA-256 digests.

The invocation path ignores the tenant's fuel, time, and memory limits and uses the module runtime's defaults. It also ignores the tenant's networking flag. The module runtime is configured without networking, so this path rejects network imports for every tenant. The concurrency limit is enforced.

### C. Module admission and invocation

A module is uploaded as text or binary. The server validates it by compiling it; a module that fails to compile is rejected and not stored. On success, the server stores the original bytes in SQLite, caches the compiled module, and returns an ID.

Figure 11 shows an invocation. Parameters are scalars or buffers. Buffers are written into guest memory starting at offset 1024. A buffer that does not fit returns HTTP 413. After the call, the server writes the tenant, module, export, outcome, elapsed time, and fuel used to an invocations table. It also publishes an `isolate.completed` event on a server-sent event stream filtered by tenant. Arguments, results, and guest output are not stored.

### D. Launcher

The launcher maps each language to a WASI command module and runs it with the runtime's `run_command` (Fig. 12, Table VI). This function builds a WASI context with arguments, preopened directories, and 1 MiB buffers for standard output and error. It then calls `_start` and returns the exit code.

For an interpreted language, the interpreter is the guest module, and the user's program is a file it reads from a preopened directory. A new language therefore needs no new host functions. The WASI build of CPython, for example, imports only WASI preview 1. Compiled interpreter modules are cached. According to the launcher's source, compiling the

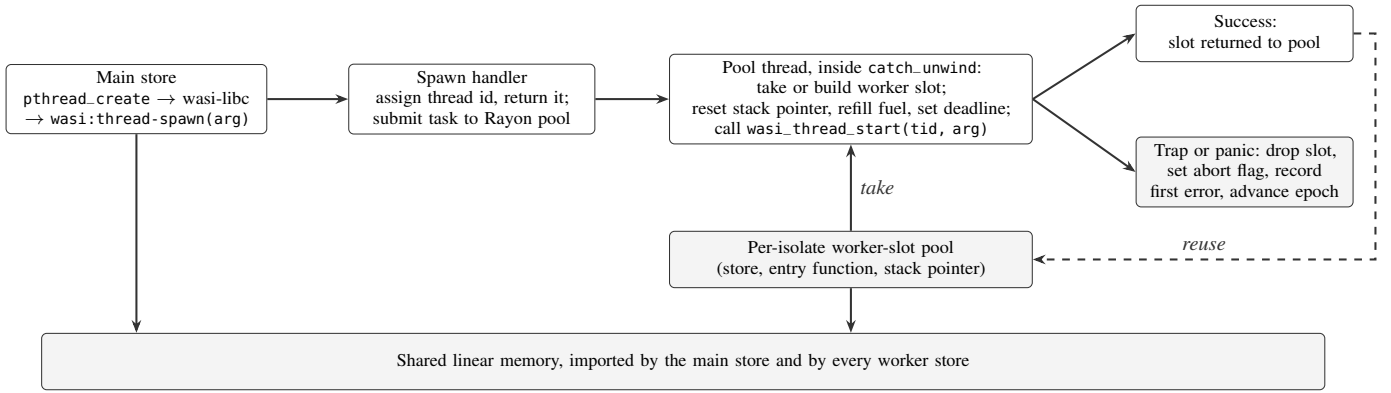


Fig. 9. Thread-spawn handler. Each guest thread runs in its own store, and all stores share one memory. A fault stays within the isolate that caused it.

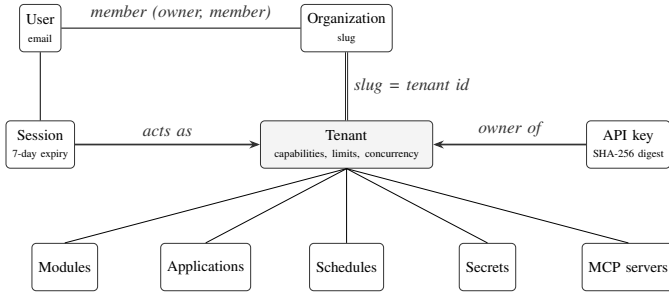


Fig. 10. Principal model. The runtime sees only tenants.

TABLE VI  
LANGUAGES SUPPORTED BY THE LAUNCHER.

| Language          | Execution                                                                                                         |
|-------------------|-------------------------------------------------------------------------------------------------------------------|
| Python, Ruby, PHP | Interpreter compiled to WASI                                                                                      |
| JavaScript        | QuickJS compiled to WASI                                                                                          |
| Node.js subset    | QuickJS with a prelude providing <code>fs</code> , <code>path</code> , <code>process</code> , <code>Buffer</code> |
| WebAssembly text  | Compiled by Wasmtime                                                                                              |
| C, Rust, Go, Zig  | Compiled to WASI by a host toolchain                                                                              |

21 MB CPython module takes about 0.5 s and instantiating the cached module about 0.1 s. Each run still gets a new store. Programs in C, Rust, Go, and Zig are compiled by host toolchains outside the sandbox on every run.

Launcher runs use capability profiles instead of the tenant flags. `compute` mounts the program directory at `/sandbox` with write access and no network. `reader` mounts it read-only. `connected` mounts it with write access and allows connections to a declared, non-empty allowlist; host names that do not resolve are dropped from the list. Python also gets its standard library, mounted read-only.

A tenant can deploy a source file or a zip bundle as an *application*. Bundles are limited to 5 MiB compressed, 20 MiB uncompressed, and 80 files. The unpacker rejects symbolic links, absolute paths, and parent-directory components. The entry point comes from an explicit argument, a `kronos.json` manifest, or a single conventional file name. Each application

has a share token. Anyone with the token can run the application without an account, subject to per-address rate limits, and can copy it as a *remix*. A remix that changes the source reverts to the compute profile with an empty allowlist. Applications can run on a fixed interval or weekly; a background task checks for due runs every 15 seconds.

### E. Agent interfaces

Figure 13 shows how agents connect in both directions. `kronos-mcp` is a JSON-RPC server over standard input and output with three tools: `kronos_deploy`, `kronos_run`, and `kronos_list_apps`. Each tool sends an HTTP request with the tenant’s API key, so an agent has exactly that tenant’s authority. The tools do not expose the raw module path, bundles, schedules, or secrets.

For outbound calls, a tenant stores secrets in a per-tenant vault and registers external MCP servers. The server makes the calls and attaches the stored secret as a bearer token, so guest code never receives the credential.

Language-model usage is metered separately from fuel. A server-wide token pool holds a total, and each tenant may use a share weighted by its tickets. Guests draw from the pool with `debit_tokens`, which fails when the tenant’s share or the pool is used up.

## IX. EVALUATION

### A. Methodology

The data come from two benchmark runs committed about an hour apart on April 28, 2026: run A at 2b3bdd40a and run B at 9d58f5da5. They measure the runtime as it was on that date, three months before the version described in Sections IV–VIII. The timer readings fall on multiples of 41.67 ns, the monotonic-clock resolution of Apple Silicon Macs, which identifies the platform. The core count was not recorded. Kronos is compared with the Wasmtime Go embedding and `wazero` [12], each measured by a Go harness of the same design. Each data point is the mean of two to eight trials, so small differences are not significant. The Kronos throughput harness drops each tenant’s runtime after creating its instance. This stops the runtime’s epoch thread, so no deadline can expire during the timed calls, although the calls still perform epoch checks.

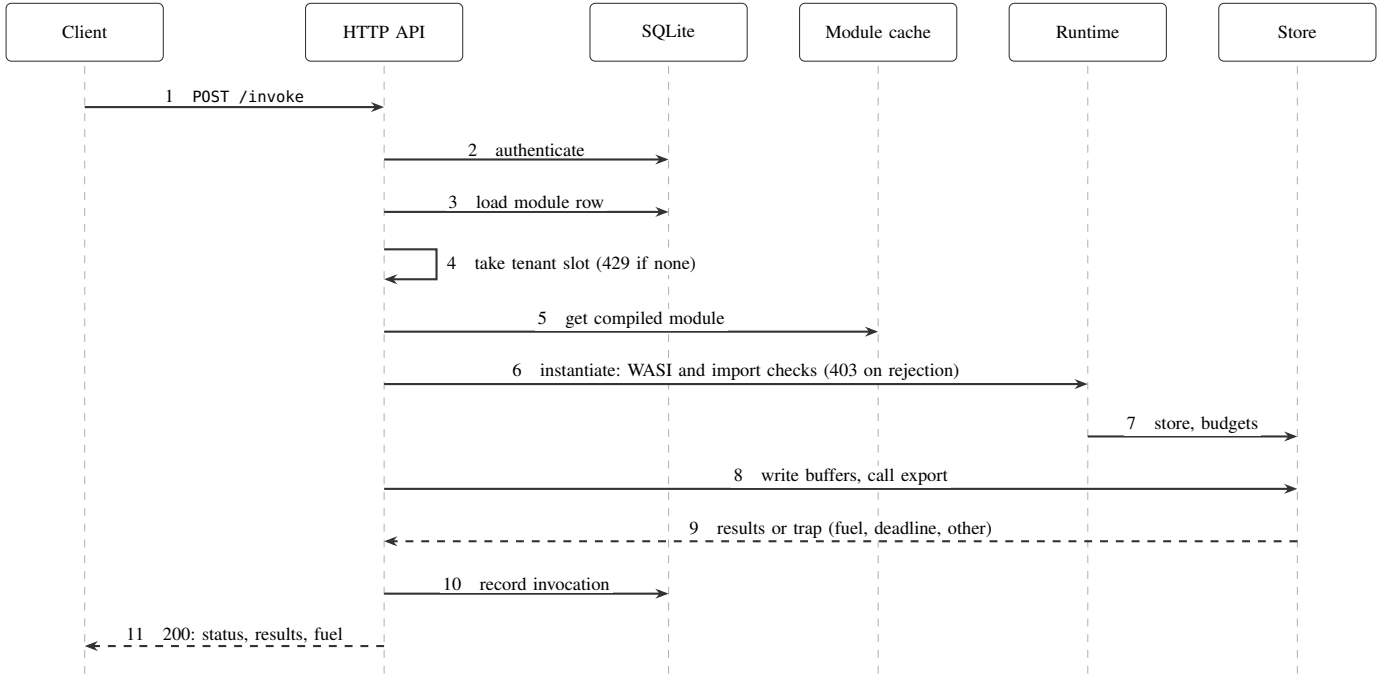


Fig. 11. Sequence of a module invocation. Steps 2–6 can end the request with 401, 403, 404, or 429 before any guest code runs.

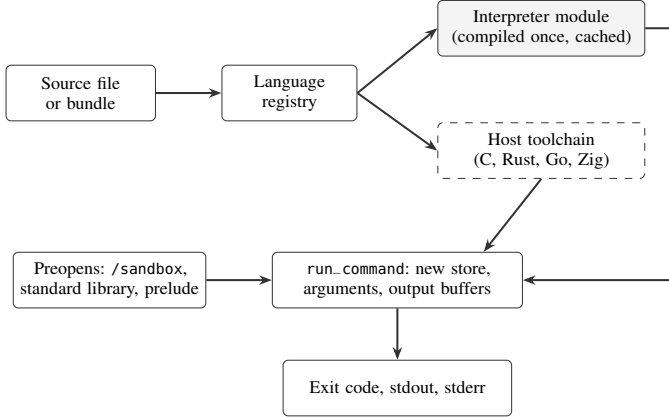


Fig. 12. Launcher execution path. Compiled languages are built on the host, outside the sandbox.

### B. Memory density

Each harness creates  $N$  isolates. An isolate is a runtime, a compiled module, and an instance of a small recursive Fibonacci module. The harness reports the growth of the process’s resident set size. Table VII gives the growth per added isolate between  $N = 100$  and  $N = 200$ , which excludes one-time costs.

The Wasmtime Go embedding used 1.4 to 1.5 times as much memory per isolate as Kronos, and wazero used about 15 times as much. Resident set size counts only pages that have been touched. The harnesses also differ in language and allocator, and the test module is small enough that fixed overhead dominates. The result measures the overhead Kronos adds to each Wasmtime store. It does not predict how many

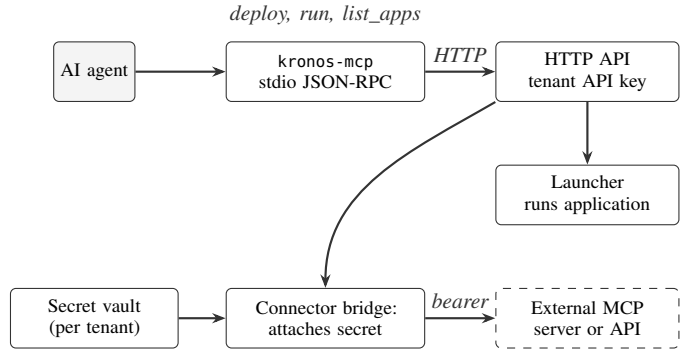


Fig. 13. Agent integration: inbound tool calls through kronos-mcp and outbound calls through the connector bridge.

TABLE VII  
RESIDENT MEMORY PER ADDED ISOLATE (MiB),  $N = 100$  TO 200.

| Runtime                 | Run A | Run B |
|-------------------------|-------|-------|
| Kronos                  | 0.15  | 0.15  |
| Wasmtime (Go embedding) | 0.23  | 0.21  |
| wazero                  | 2.19  | 2.22  |

realistic workloads fit on a machine.

### C. Multi-tenant throughput

$N$  threads each call `sfib(20)` once on their own isolate, for  $N$  from 1 to 1024, with three trials per point. Table VIII reports each runtime’s peak rate.

Kronos peaked 7 to 11 times lower than the Wasmtime Go embedding and 11 to 12 times lower than wazero. With

TABLE VIII  
PEAK BURST THROUGHPUT (CALLS/S), WITH  $N$  AT THE PEAK.

| Runtime       | Run A         | Run B         |
|---------------|---------------|---------------|
| Kronos        | 14,365 (64)   | 14,435 (64)   |
| Wasmtime (Go) | 103,237 (512) | 164,388 (512) |
| wazero        | 170,637 (256) | 157,207 (256) |

TABLE IX  
MATRIX MULTIPLY UNDER KRONOS, RUN B (MEAN MS).

| $N$ | Intrinsic | Threaded (4) | Plain |
|-----|-----------|--------------|-------|
| 32  | 7.11      | 1.49         | 0.023 |
| 64  | 34.7      | 5.56         | 0.151 |
| 96  | 116.8     | 18.5         | 0.503 |
| 128 | 277.6     | 42.0         | 4.91  |

one tenant the gap was smaller. Kronos handled 7,529 and 7,837 calls per second, Wasmtime 8,811 and 6,540, and wazero 23,392 and 21,486. The Kronos peak varied by less than 1% between runs; the Wasmtime peak varied by 59%. Design notes from April attribute the multi-tenant overhead to per-call intrinsic registration and budget resets. The invocation path was not profiled, so this explanation is unconfirmed.

#### D. Threaded execution

Run B timed an  $N \times N$  double-precision matrix multiply in three builds (Table IX). The *intrinsic* build uses the walk wrappers without threads and calls the host about three times per inner element. The *threaded* build compiles the same source with `-pthread` and runs four guest threads, with the inner loops inside the guest. The *plain* build is a hand-written single-threaded module with no host calls.

The threaded build was 4.8 to 6.6 times faster than the intrinsic build and 8.6 to 65 times slower than the plain build. Most of the gap between the intrinsic and threaded builds comes from removing host calls. These runs cannot separate any gain from the extra threads. They do show that threaded guests run to completion through the custom spawn handler. From  $N = 96$  to  $N = 128$  the plain module’s time grew 9.8 times, while the arithmetic grew 2.4 times. This result is unexplained.

## X. ANALYSIS

Every instantiation path, host handler, and configured limit at commit 2b73f3cb8 was checked against the properties in Section II-B. The analysis is a code review. No exploit modules were built.

#### A. Enforced properties

- On the server’s module path, the import check runs before the store is created. A module that imports an ungranted file or network function runs no instructions (P1).
- Each invocation uses a new store, and no mutable runtime state carries over between invocations. Compiled code,

including cached interpreters, is shared, but no mutable state is.

- The spawn handler turns a trap or panic in a guest thread into an error for that isolate and does not stop the process. A unit test covers this case.
- Interpreted languages run as unmodified WASI modules, so adding a language does not enlarge the import surface.
- A remix that changes an application’s source loses the network allowlist.

#### B. Instantiation entry points

Three entry points in Table V weaken or skip the check.

- The fast path in `invoke`, used for modules that import only functions, does not run the check. A module that defines its own memory and imports `socket` and `connect` instantiates and runs. This path does not record guest memory for the host handlers, so handlers that read guest memory return EIO. `socket` and `close` read no memory and succeed.
- `instantiate` creates a default WASI context for any module that imports WASI. When a context exists, the check treats the filesystem as granted. A module that imports one WASI function and `openat` therefore passes the check whatever the tenant holds.
- `instantiate_threaded` and `run_command` also treat the filesystem as granted.

The server’s module path uses none of these entry points. All three are public library functions, however, and the command-line tool uses the fast path. No test checks that a guest module is rejected. The only rejection test calls the path resolver directly, and the demonstration programs print their results without assertions. P1 therefore holds only on the server’s module path. Routing every caller through one checked instantiation function, with a rejection test for each entry point, would close this gap.

#### C. Unenforced limits

Several configured limits are not enforced.

- *Memory*. The maximum applies only under the pooling allocator, which is off by default (Section VI).
- *Per-tenant limits*. The invocation path does not read the tenant’s fuel, time, or memory limits.
- *Threads*. Each spawned thread gets full fuel and a new deadline, so a threaded isolate’s budget grows with each thread it creates (P3).
- *Host calls*. A socket read can block for five seconds against a two-second deadline, and host work uses no fuel. The read and `recvfrom` handlers allocate a host buffer of the size the guest requests, up to about 2 GiB, before checking it against guest memory. Several handlers add a guest pointer to a length without checking for overflow.

#### D. The epoch counter as an abort signal

The spawn handler stops an isolate’s threads by advancing the engine epoch by 2,001 ticks. Every store on the engine reads that epoch. Any running store whose deadline falls

within those ticks traps at its next check (Fig. 7). With the default deadline, that is every running store. If tenants share a runtime, one tenant’s faulty threaded module stops the running calls of all tenants, which violates P2. The server shares one runtime but does not use the threaded path, and the benchmarks use one runtime per tenant, so neither setup shows the fault. One runtime per tenant contains it, at the cost of a 1 kHz ticker thread per tenant and, with pooling, a separate memory reservation. An abort signal scoped to the isolate avoids both costs.

#### E. File mediation

The file handlers resolve paths lexically and follow symbolic links, while two source comments describe the paths as canonicalized. No code outside the tests sets a tenant’s root directory, so every Tier 3 file call returns ENOENT in deployment. Deployed programs reached files only through WASI preopened directories in the launcher.

#### F. Stale comments

Several comments at this commit describe removed or changed designs. They call the intrinsics Rayon-backed after they became sequential, give the host function count as about 30 instead of 56, and refer to a configuration field that no longer exists. They do not change behavior.

#### G. Limitations of this study

The security analysis is a code review without exploit construction. The measurements come from two runs on one machine, use one small module and harnesses in different languages, and cover the April code rather than the July version. Kronos runs only WebAssembly guests and cannot run arbitrary Linux programs or native extensions.

### XI. SUBSEQUENT DEVELOPMENT

After July 27, 2026, the research components were removed from the code base. The intrinsics, threaded execution, and name-gated Tier 3 handlers were deleted, and WASI preopened directories became the only form of file authority. Preopens bind authority to a specific resource when it is granted, which matches the exokernel notion of a secure binding [10]. C++ support returned by running clang, compiled to WebAssembly, inside the sandbox. The successor system, kronoslabs.dev, fixes its guest interface at 40 env functions and the 46 WASI functions, and a test checks that count.

### XII. RELATED WORK

1) *Sandboxes for untrusted code*: Firecracker and gVisor isolate arbitrary Linux programs, using a microVM and a user-space kernel respectively [1], [2]. Services that run model-generated code build on such systems [3]–[5]. They can run any Linux program. Kronos runs only WebAssembly, which limits what it can run but allows an import check. Cloudflare Workers run many tenants as V8 isolates in one process, a process model close to that of Kronos [16].

2) *WebAssembly outside the browser*: Lucet instantiated WebAssembly modules in microseconds with kilobytes of overhead per instance [17]. Faasm isolates serverless functions with WebAssembly memory safety and operating-system controls [18]. RLBox sandboxes libraries in Firefox with WebAssembly [19]. WaVe formally verifies that a WASI runtime’s host calls preserve memory safety and restrict file and network access [20]; Kronos’s Tier 3 handlers aimed at the same property without verification. Yu et al. show that WebAssembly tenants can exhaust shared host resources [21], which the unenforced limits in Section X permit.

3) *Capabilities*: Capability-based protection dates to Dennis and Van Horn [23]. Capsicum added a capability mode to UNIX [24], and object-capability systems pass authority as explicit references [25]. The Kronos name gate is a coarse capability layer over import names. WASI preopens, used by its successor, bind authority to specific resources.

### XIII. CONCLUSION

Kronos used the import list of a WebAssembly module for multi-tenant access control. It combined a capability check at instantiation with per-invocation stores bounded by fuel and epoch deadlines. On the server’s module path, the check kept modules with ungranted imports from running, new stores isolated state between calls, and a custom spawn handler kept thread faults within their isolate. The analysis found three weaknesses. The check held only on entry points that called it, several configured limits were not enforced, and an engine-wide epoch counter cannot serve as a per-isolate abort signal. In comparable harnesses, Kronos added less memory per isolate than the Wasmtime Go embedding and wazero, and its burst throughput was much lower. Its successor kept per-invocation stores and replaced name-based gating with WASI preopened directories.

### ACKNOWLEDGMENT

Ajax Li contributed to the benchmark harnesses and the evaluation runs.

### AVAILABILITY

The source code and raw benchmark outputs are in a private repository and are available from the author on request. The architecture described here is at commit 2b73f3cb8. The measurements in Section IX are from commits 2b3bdd40a and 9d58f5da5.

### REFERENCES

- [1] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *Proc. USENIX NSDI*, 2020, pp. 419–434.
- [2] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “The true cost of containing: A gVisor case study,” in *Proc. USENIX HotCloud*, 2019.
- [3] E2B, “E2B,” <https://e2b.dev/>. Accessed Sep. 12, 2026.
- [4] Modal Labs, “Sandboxes,” Modal documentation. <https://modal.com/docs/guide/sandboxes>. Accessed Sep. 12, 2026.
- [5] Daytona, “Secure infrastructure for running AI-generated code,” <https://www.daytona.io/>. Accessed Sep. 12, 2026.

- [6] R. Marchand, A. O Cathain, J. Wynne, P. M. Giavridis, S. Jennings, F. Tuxworth, T. H. Dur, S. Deverett, J. Wilkinson, J. Gwartz, and H. Coppock, “Quantifying frontier LLM capabilities for container sandbox escape,” arXiv:2603.02277, 2026.
- [7] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. F. Bastien, “Bringing the web up to speed with WebAssembly,” in *Proc. ACM PLDI*, 2017, pp. 185–200.
- [8] Bytecode Alliance, “Security,” Wasmtime documentation. <https://docs.wasmtime.dev/security.html>. Accessed Sep. 12, 2026.
- [9] WebAssembly Community Group, “WASI design principles.” <https://github.com/WebAssembly/WASI/blob/main/docs/DesignPrinciples.md>. Accessed Sep. 12, 2026.
- [10] D. R. Engler, M. F. Kaashoek, and J. O’Toole, Jr., “Exokernel: An operating system architecture for application-level resource management,” in *Proc. ACM SOSP*, 1995, pp. 251–266.
- [11] Bytecode Alliance, “Wasmtime.” <https://wasmtime.dev/>. Accessed Sep. 12, 2026.
- [12] “wazero.” <https://wazero.io/>. Accessed Sep. 12, 2026.
- [13] M. Frigo, C. E. Leiserson, and K. H. Randall, “The implementation of the Cilk-5 multithreaded language,” in *Proc. ACM PLDI*, 1998, pp. 212–223.
- [14] “Rayon: A data parallelism library for Rust.” <https://github.com/rayon-rs/rayon>. Accessed Sep. 12, 2026.
- [15] WebAssembly Community Group, “wasi-threads.” <https://github.com/WebAssembly/wasi-threads>. Accessed Sep. 12, 2026.
- [16] K. Varda, “Mitigating Spectre and other security threats: The Cloudflare Workers security model,” Cloudflare Blog, Jul. 29, 2020.
- [17] P. Hickey, “Lucet takes WebAssembly beyond the browser,” Fastly Blog, Mar. 28, 2019.
- [18] S. Shillaker and P. Pietzuch, “Faasm: Lightweight isolation for efficient stateful serverless computing,” in *Proc. USENIX ATC*, 2020, pp. 419–433.
- [19] S. Narayan, C. Disselkoen, T. Garfinkel, N. Froyd, E. Rahm, S. Lerner, H. Shacham, and D. Stefan, “Retrofitting fine grain isolation in the Firefox renderer,” in *Proc. USENIX Security*, 2020, pp. 699–716.
- [20] E. Johnson, E. Laufer, Z. Zhao, D. Gohman, S. Narayan, S. Savage, D. Stefan, and F. Brown, “WaVe: A verifiably secure WebAssembly sandboxing runtime,” in *Proc. IEEE S&P*, 2023, pp. 2940–2955.
- [21] Z. Yu, D. Zhan, L. Ye, H. Yu, H. Zhang, and Z. Tian, “Exploring and exploiting the resource isolation attack surface of WebAssembly containers,” in *Proc. USENIX Security*, 2025, pp. 1111–1128.
- [22] S. Narayan, C. Disselkoen, D. Moghimi, S. Cauligi, E. Johnson, Z. Gang, A. Vahldiek-Oberwagner, R. Sahita, H. Shacham, D. Tullsen, and D. Stefan, “Swivel: Hardening WebAssembly against Spectre,” in *Proc. USENIX Security*, 2021, pp. 1433–1450.
- [23] J. B. Dennis and E. C. Van Horn, “Programming semantics for multiprogrammed computations,” *Commun. ACM*, vol. 9, no. 3, pp. 143–155, 1966.
- [24] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, “Capsicum: Practical capabilities for UNIX,” in *Proc. USENIX Security*, 2010.
- [25] M. S. Miller, “Robust composition: Towards a unified approach to access control and concurrency control,” Ph.D. dissertation, Johns Hopkins University, 2006.